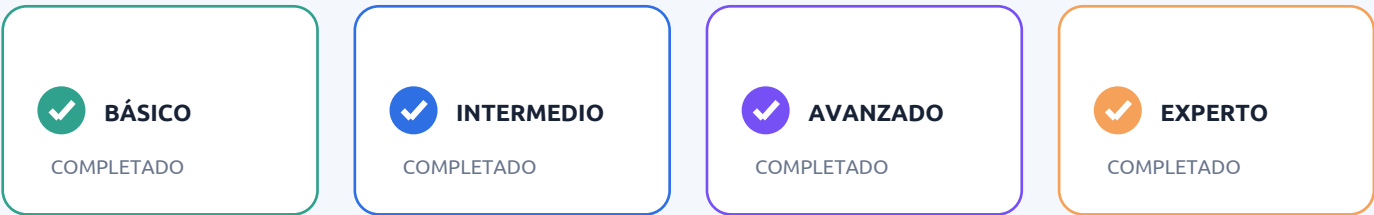
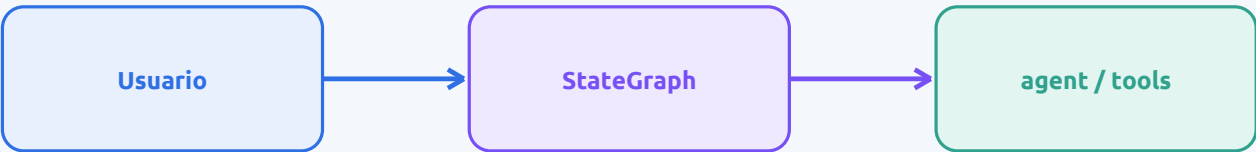
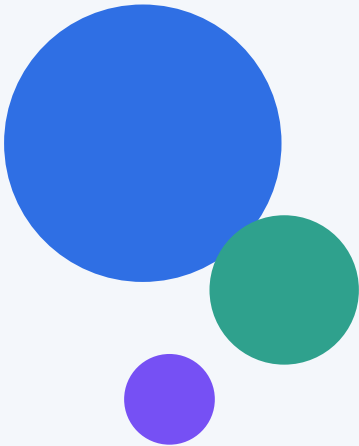


NIVEL EXPERTO

LangGraph · Arquitectura de agentes · Testing · Evaluación

De un agent loop manual a una arquitectura basada en estado, nodos y transiciones



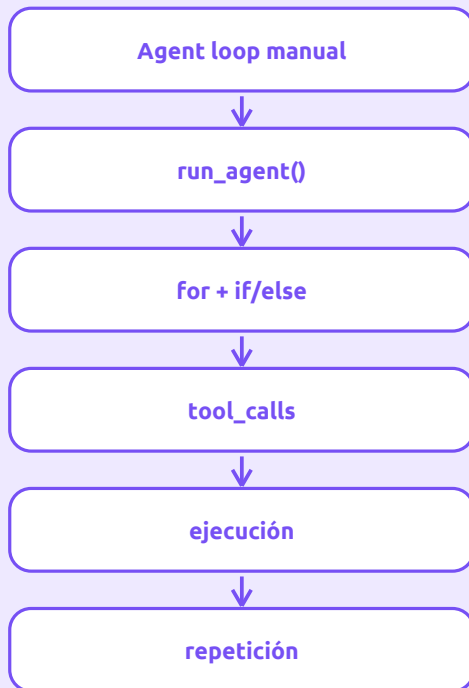
ESTADO · NODOS · TRANSICIONES · EVALUACIÓN

Arquitectura modular · seguridad por capas · logging privado · pruebas reproducibles

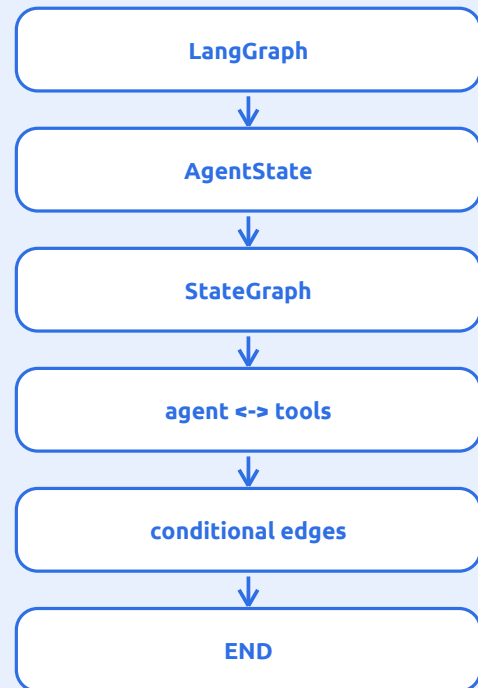
1. ¿Qué cambia en el Nivel Experto?

La misma lógica agentic se representa ahora como estado, nodos y rutas explícitas.

NIVEL AVANZADO



NIVEL EXPERTO



IDEA PRINCIPAL

LangGraph no se eligió por ser universalmente mejor.

Primero se construyó el flujo manual para comprender estado, llamadas al modelo, tool calls, resultados, repetición, límites y finalización.

Después se introdujo el framework para comparar qué parte de esa orquestación puede representar mediante nodos y transiciones.

2. ¿Por qué LangGraph?

Una decisión de ingeniería tomada después de entender el mecanismo manual.

IMPLEMENTACIÓN MANUAL

VENTAJAS

- control explícito
- pocas abstracciones
- aprendizaje del funcionamiento interno
- menor dependencia del framework

INCONVENIENTES

- `run_agent()` concentra responsabilidades
- routing manual
- crecimiento progresivo de condicionales
- más difícil escalar estados y rutas

LANGGRAPH

VENTAJAS

- estado explícito
- nodos separados
- transiciones visibles
- routing declarativo
- arquitectura extensible
- responsabilidades desacopladas

INCONVENIENTES

- nueva dependencia
- conceptos adicionales
- mayor estructura inicial
- puede ocultar comportamiento si no se entiende

CONCLUSIÓN

LangGraph se adoptó después de comprender el flujo manual.

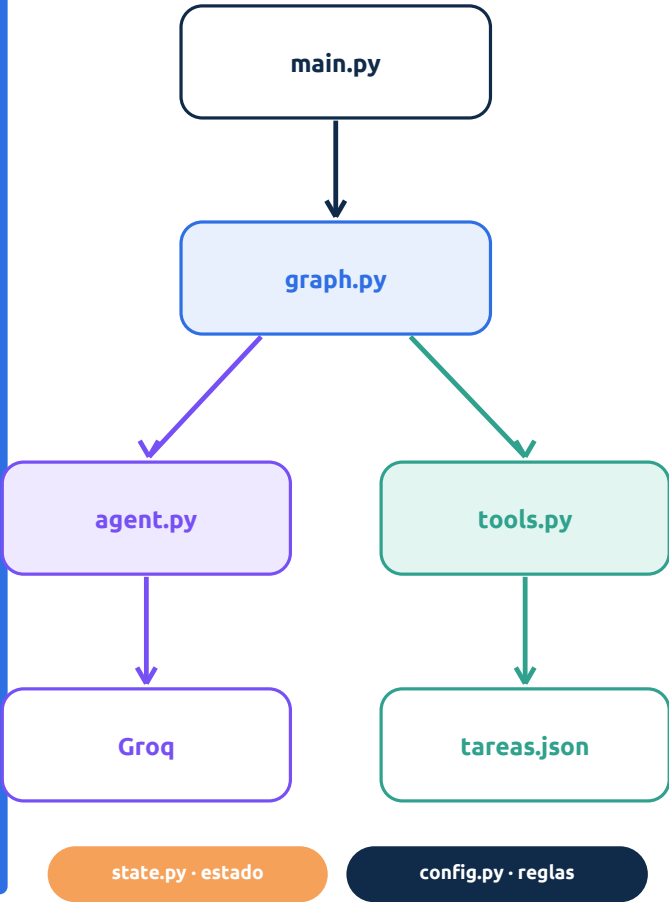
3. Arquitectura del Nivel Experto

Siete módulos pequeños, responsabilidades separadas y dependencias visibles.

ÁRBOL REAL

```
src/experto/  
|- __init__.py  
|- agent.py  
|- config.py  
|- graph.py  
|- main.py  
|- state.py  
`- tools.py
```

RELACIONES



RESPONSABILIDADES

config.py
modelo, rutas, logging y límite no sensible

tools.py
schemas, validación y despacho seguro

graph.py
nodos, routing y StateGraph

state.py
estructura del estado compartido

agent.py
adaptador Groq y normalización

main.py
terminal y ejecución

4. El estado del agente

Los nodos comparten información sin concentrar toda la lógica en una sola función.

AgentState

```
AgentState {  
  
    messages  
  
    agent_steps  
  
    final_answer  
  
    error  
  
}
```

messages

Historial necesario para la ejecución actual.

agent_steps

Número de llamadas o intentos del nodo agent.

final_answer

Respuesta natural final preparada para mostrar.

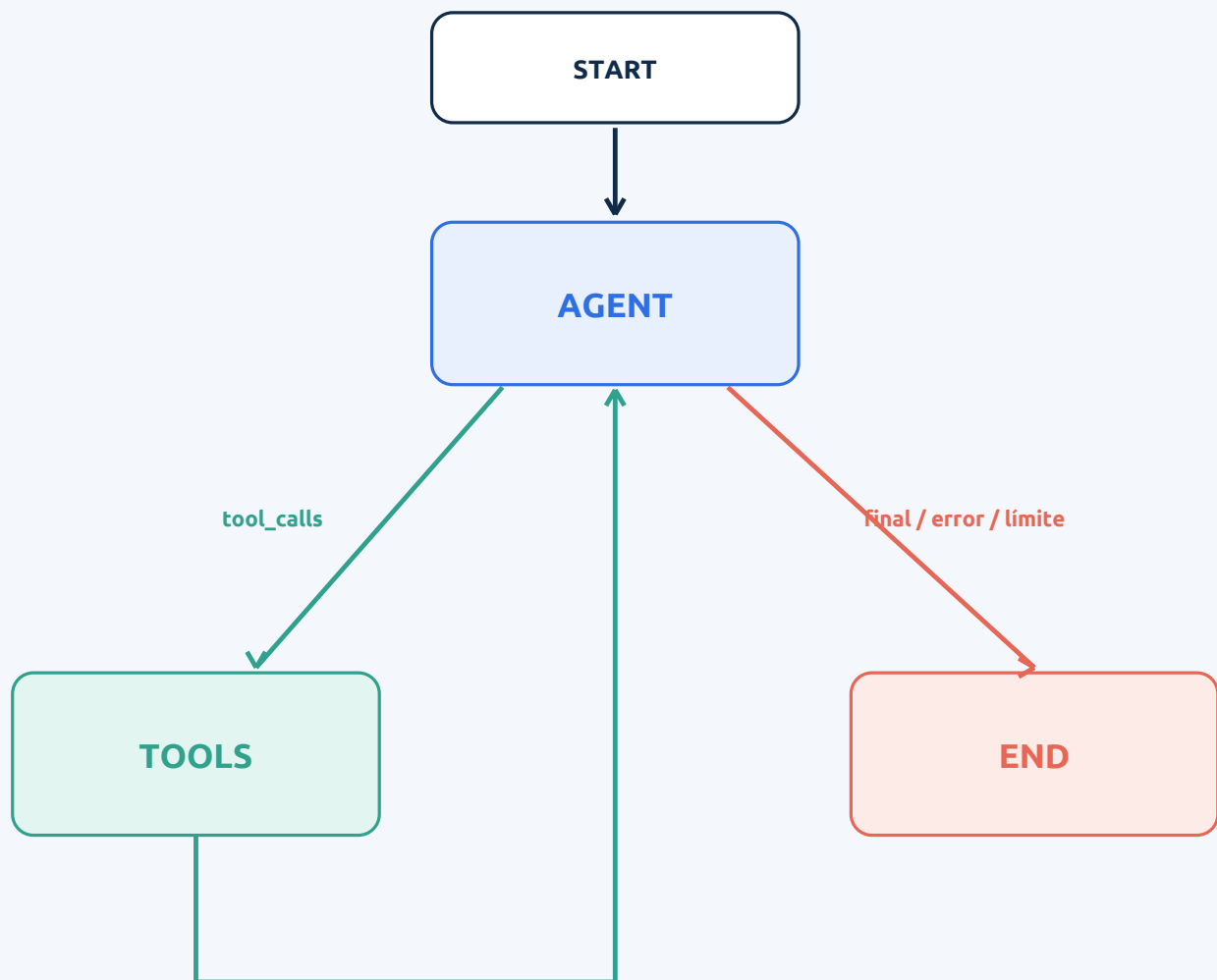
error

Error controlado que puede finalizar el grafo.

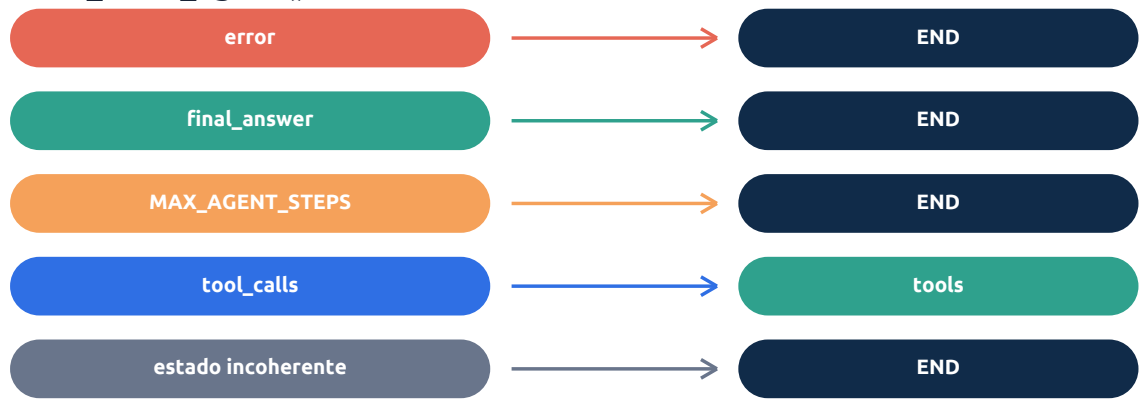
IDEA CLAVE • Los nodos reciben estado y producen un nuevo estado.

5. StateGraph

El routing decide si ejecutar herramientas o terminar de forma segura.



route_after_agent()



6. Nodo agent

El nodo consulta al modelo, normaliza la respuesta y actualiza el estado.

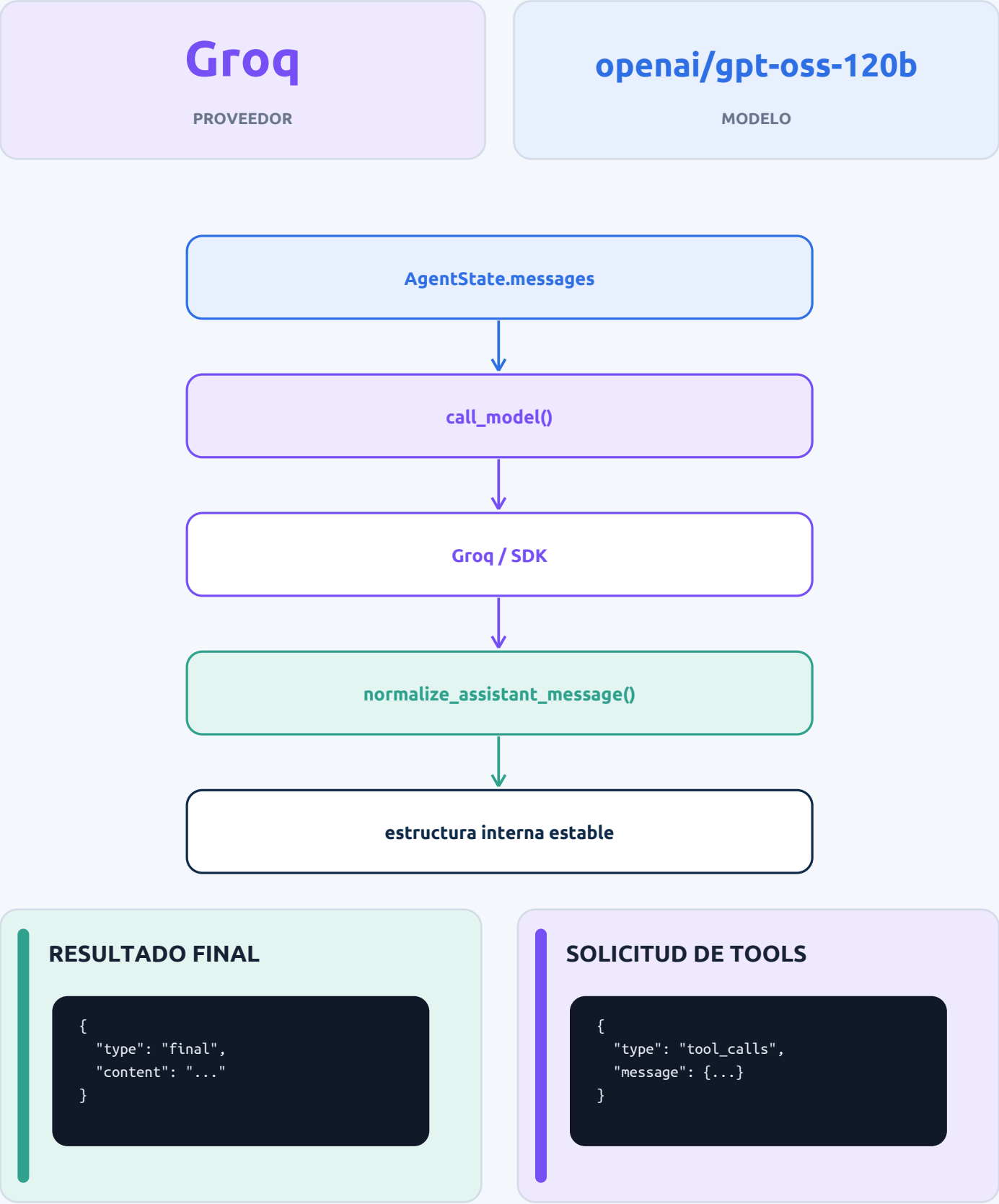
agent_node()



EL NODO AGENT NO EJECUTA HERRAMIENTAS

7. Adaptador Groq

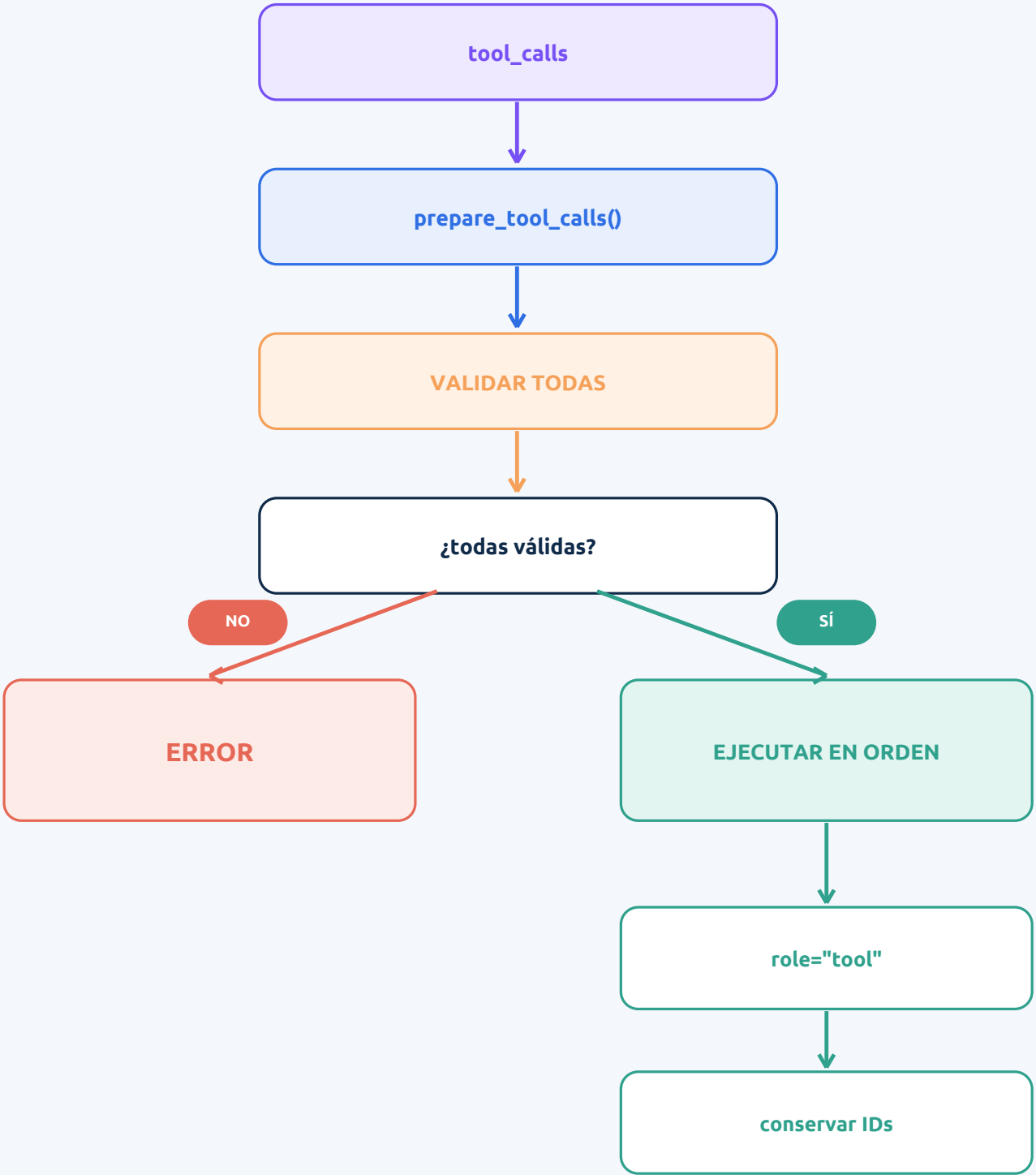
agent.py transforma objetos del SDK en una estructura interna estable.



La normalización desacopla parcialmente el grafo de los objetos concretos del SDK.

8. Nodo tools

Primero valida la ronda completa; solo después ejecuta en orden.



SEGURIDAD · Ninguna herramienta se ejecuta hasta validar todas las solicitudes de la ronda.

9. Dos herramientas locales

Un catálogo pequeño, cerrado y validado para necesidades concretas.

calculate

add · subtract · multiply · divide

PROTECCIONES

- operación permitida
- JSON válido
- argumentos exactos
- tipos numéricos
- bool rechazado como número
- números finitos
- división entre cero controlada

```
calculate("multiply", 90, 3)

result = 270.0
```

get_task_info

fuentes · data/tareas.json

DEVUELVE

- nombre
- prioridad
- duración
- categoría

CONTROLA

- fichero inexistente
- JSON inválido
- estructura incorrecta
- tarea inexistente

```
get_task_info("estudiar Python")

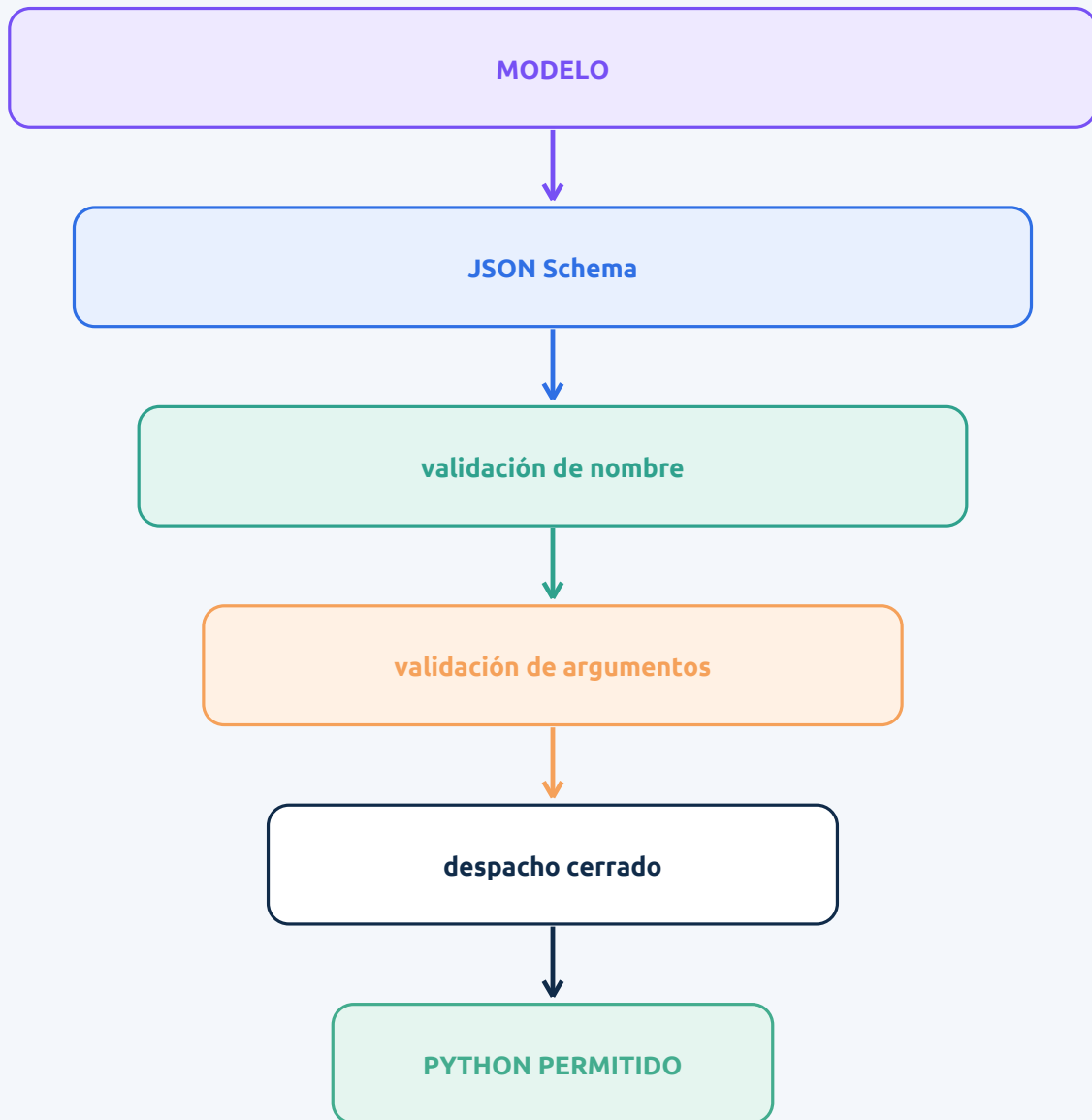
90 min · alta · estudio
```

BÚSQUEDA NORMALIZADA

El nombre se compara sin distinguir mayúsculas y tras eliminar espacios exteriores; los datos siguen siendo locales.

10. Seguridad del tool calling

El LLM propone una acción; Python decide si está permitida.



NO EXISTE EJECUCIÓN ARBITRARIA

✗ eval()

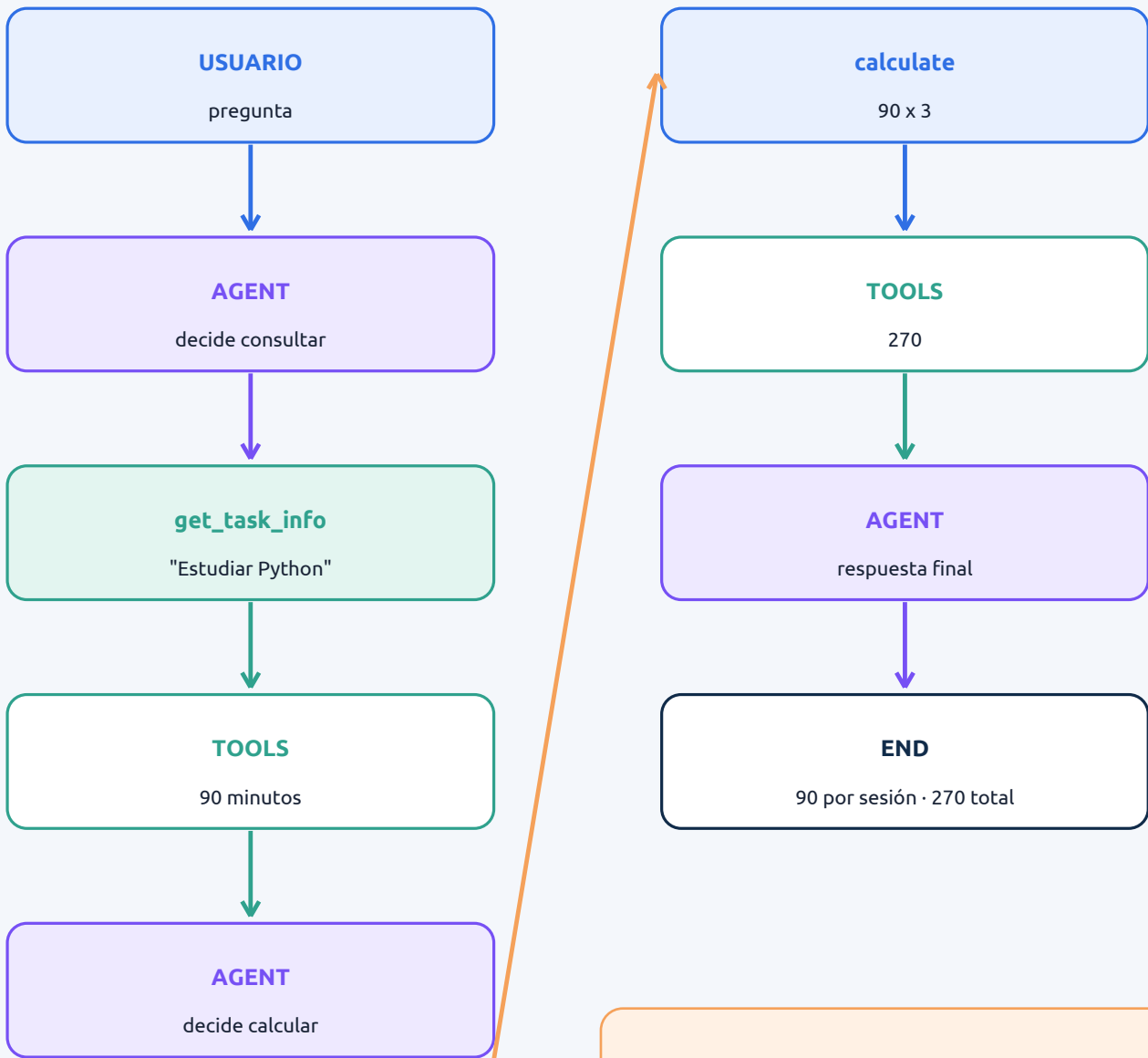
✗ exec()

✗ getattr() dinámico

✗ imports dinámicos

11. Ejemplo de flujo multipaso

Pregunta: ¿Cuánto dura estudiar Python y cuántos minutos serían si hiciera esa tarea 3 veces?



SECUENCIA VARIABLE

El modelo puede calcular directamente algunos resultados sin solicitar calculate. La secuencia exacta puede variar.

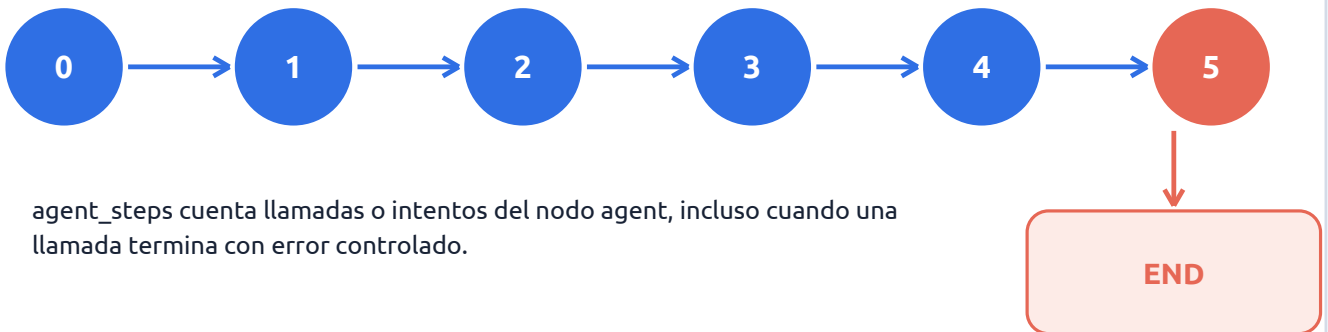
12. Límite de seguridad

Un contador semántico evita bucles indefinidos de tool calling.

MAX_AGENT_STEPS = 5

LÍMITE CONFIGURABLE

agent_steps



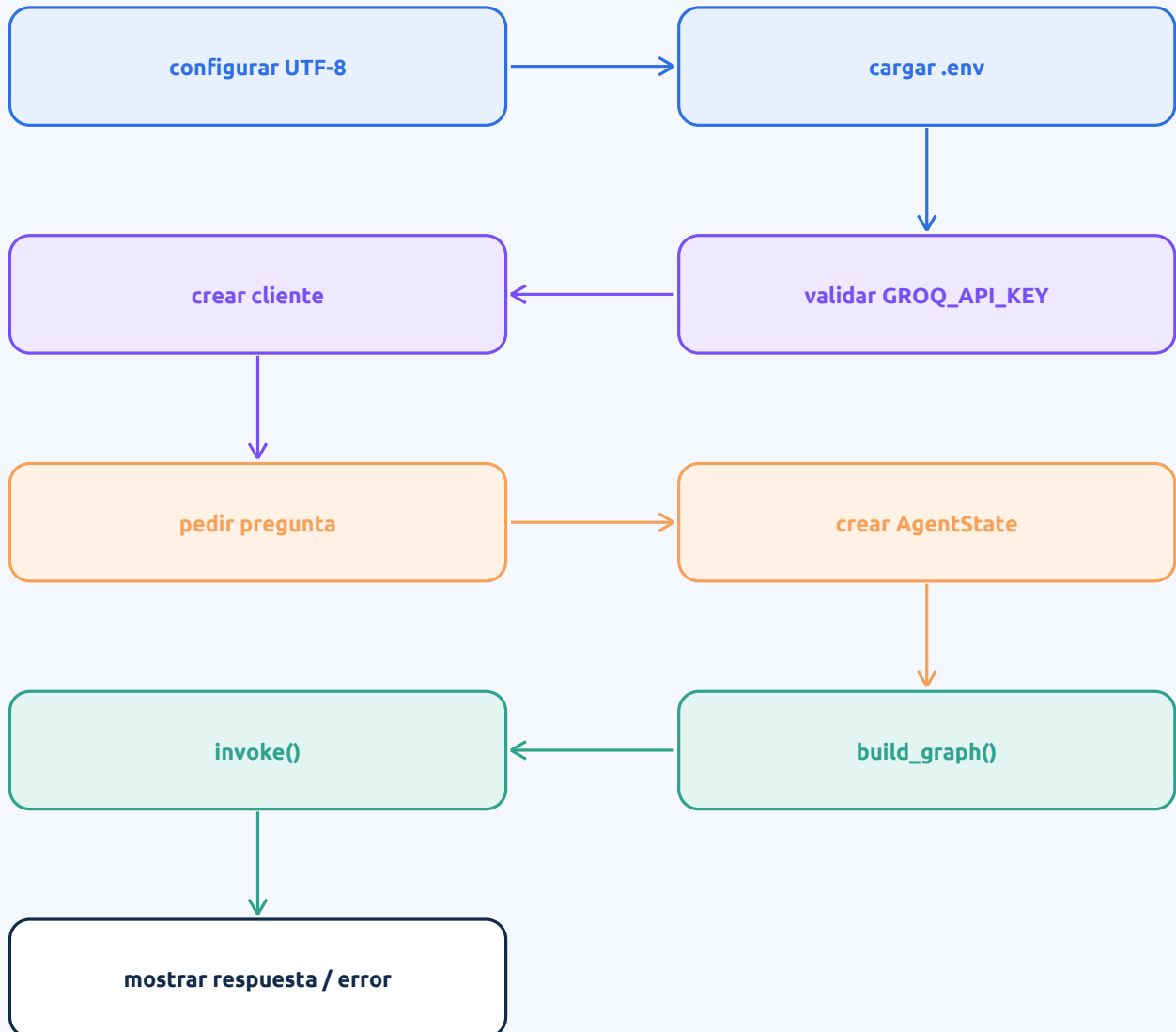
NO ES SIMPLEMENTE `recursion_limit`

Es un límite propio del dominio: cuenta semánticamente los intentos del agente y detiene nuevas llamadas al modelo.

13. CLI del Nivel Experto

main.py prepara el entorno, invoca el grafo y presenta el resultado.

```
python -m src.experto.main
```



main.py no ejecuta herramientas ni llama directamente a Groq.

14. UTF-8 en Windows

Una incidencia real de consola se convirtió en una protección defensiva.

INCIDENCIA

U+202F

Una respuesta contenía un espacio estrecho de no separación. La terminal cp1252 no podía representarlo correctamente.

respuesta Groq



terminal cp1252

SOLUCIÓN DEFENSIVA

```
sys.stdout.reconfigure(encoding="utf-8")  
sys.stderr.reconfigure(encoding="utf-8")
```

Antes se comprueba que `reconfigure()` exista y sea invocable. Si el entorno no lo permite, la aplicación continúa sin romperse.

Resultado: salida Unicode legible y arranque tolerante a distintos entornos.

15. Observabilidad sin sacrificar privacidad

Eventos técnicos suficientes para diagnosticar, sin conservar conversaciones completas.

Consola

DESTINO 1

logs/app.log

DESTINO 2

fecha | nivel | módulo | evento

SE REGISTRA

- inicio
- entrada en nodos
- routing
- herramienta seleccionada
- finalización
- errores controlados

NO SE REGISTRA

- GROQ_API_KEY
- preguntas o respuestas completas
- messages
- argumentos JSON completos
- tareas
- cuerpos HTTP

LOCAL

UTF-8

APPEND

IGNORADO POR GIT

Si el archivo no está disponible, la aplicación sigue con la consola.

16. Tests automáticos

Una suite determinista comprueba herramientas, routing y adaptación del modelo.

64

TESTS

100%

PASS HISTÓRICO

64 passed

test_tools.py

herramientas, JSON, tipos, números finitos, errores y despacho

test_graph.py

routing, tool calls, IDs, varias tools, validación total y límites

test_agent.py

normalización, cliente simulado, parámetros, errores y estado

Los tests aumentan la confianza; no garantizan que el sistema sea perfecto.

17. Evaluación funcional

Seis preguntas reales, una ejecución por caso y un resultado histórico conservado.



CASOS

✓ respuesta_directa	PASS	✓ calculadora	PASS
✓ consulta_prioridad	PASS	✓ consulta_duracion	PASS
✓ multipaso	PASS	✗ tarea_inexistente	FAIL

POR QUÉ FALLÓ tarea_inexistente

- El agente indicó correctamente que no encontró la tarea.
- No inventó una prioridad.
- El validador textual no reconoció la expresión "no he encontrado".
- Por tanto, el FAIL histórico es un falso negativo; no se cambia a 100%.

18. Pytest vs evaluación funcional

Ambas medidas son necesarias, pero responden a preguntas distintas.

PYTEST

¿Nuestro código se comporta como esperamos?

- determinista
- mocks y simulaciones
- funciones
- validaciones
- routing
- errores

64 / 64

TESTS

EVALUACIÓN

¿El agente completo responde bien ante preguntas reales?

- modelo real
- variabilidad
- tools
- respuesta natural
- métricas
- validadores textuales

5 / 6

CASOS

64/64 y 5/6 miden dimensiones diferentes del sistema.

19. Privacidad por diseño

Los artefactos sensibles o variables permanecen locales cuando corresponde.

.env

credencial local

IGNORADO



conversaciones/*.json

historiales

IGNORADO



logs/*.log

eventos técnicos

IGNORADO



evals/results/*.json

resultados variables

IGNORADO



.gitkeep conserva las carpetas necesarias sin versionar su contenido variable.

20. ¿Qué NO hace todavía?

Limitaciones de alcance documentadas; no son fallos del funcionamiento existente.

2 herramientas

calculate y get_task_info

Generación variable

las respuestas pueden cambiar entre ejecuciones

Sin checkpointer

LangGraph no guarda estado entre procesos

Sin memoria persistente

el Nivel Experto no conserva conversaciones

CLI independiente

cada ejecución comienza con una pregunta nueva

Validador textual

puede producir falsos negativos

El sistema es una base técnica extensible, no un producto universalmente terminado.

21. Evolución completa

Cada nivel conserva lo aprendido y añade una nueva capa de capacidad.



22. Conceptos aprendidos

Un mapa de capacidades técnicas construido de forma progresiva.

LLM Y CONVERSACIÓN

- LLM APIs
- prompts
- roles
- memoria conversacional
- persistencia

TOOLS Y CONTROL

- tool calling
- JSON Schema
- validación
- agent loops
- límites

ORQUESTACIÓN

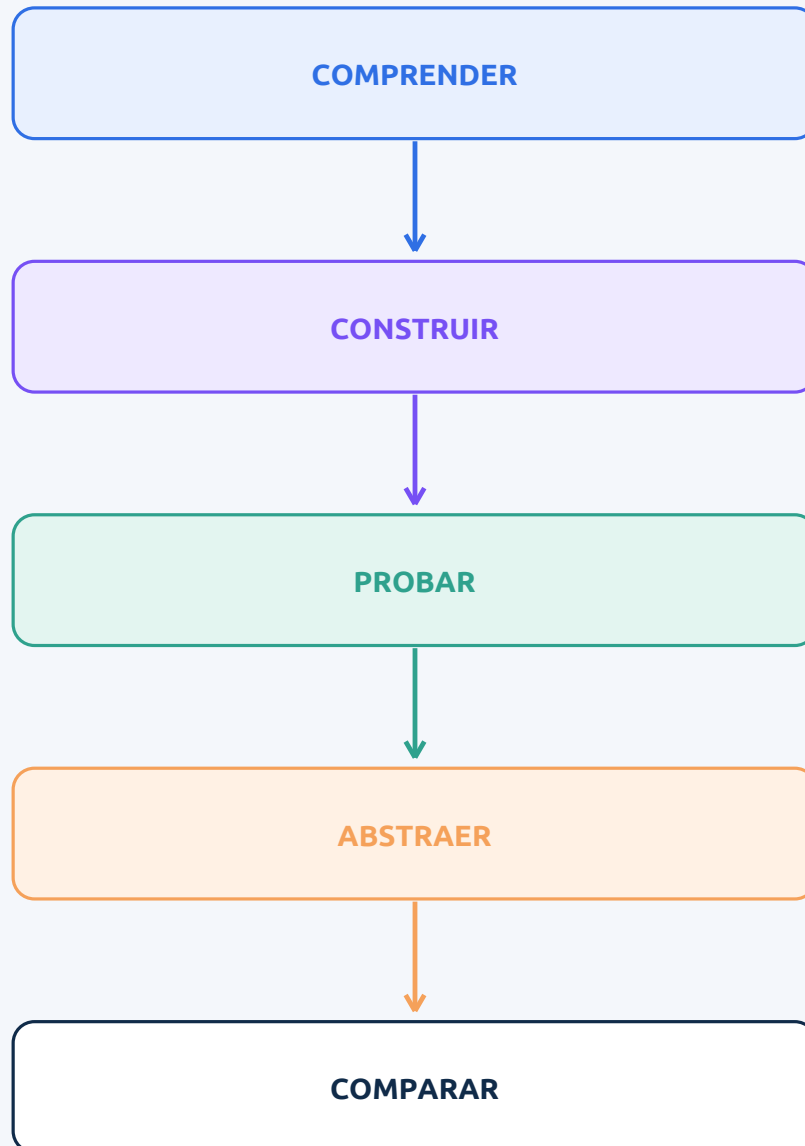
- LangGraph
- StateGraph
- nodos
- edges
- routing

CALIDAD Y PROYECTO

- seguridad
- logging
- pytest
- evaluación
- arquitectura modular
- privacidad

23. La lección principal

Construir primero el agent loop manual permitió comprender después qué abstrae LangGraph.



OBJETIVO REAL

No se trataba solo de conseguir que el agente funcionara, sino de entender por qué funciona, qué decide nuestro código, qué decide el modelo y qué responsabilidad asume el framework.

RETO COMPLETADO

De una llamada a un LLM a un agente modular, probado y evaluado.

✓ Básico COMPLETADO

✓ Intermedio COMPLETADO

✓ Avanzado COMPLETADO

✓ Experto COMPLETADO

64

TESTS

5/6

EVALUACIÓN

83.3%

ÉXITO

2

HERRAMIENTAS

MAX_AGENT_STEPS = 5

LangGraph · StateGraph · agent · tools · routing · seguridad · logging · pytest · evaluación